

Project B5 — Replicated, Strongly Consistent Key-Value Store in Go

2. Global architectural invariants

These rules apply across every component and must never be violated by generated code:

MUST

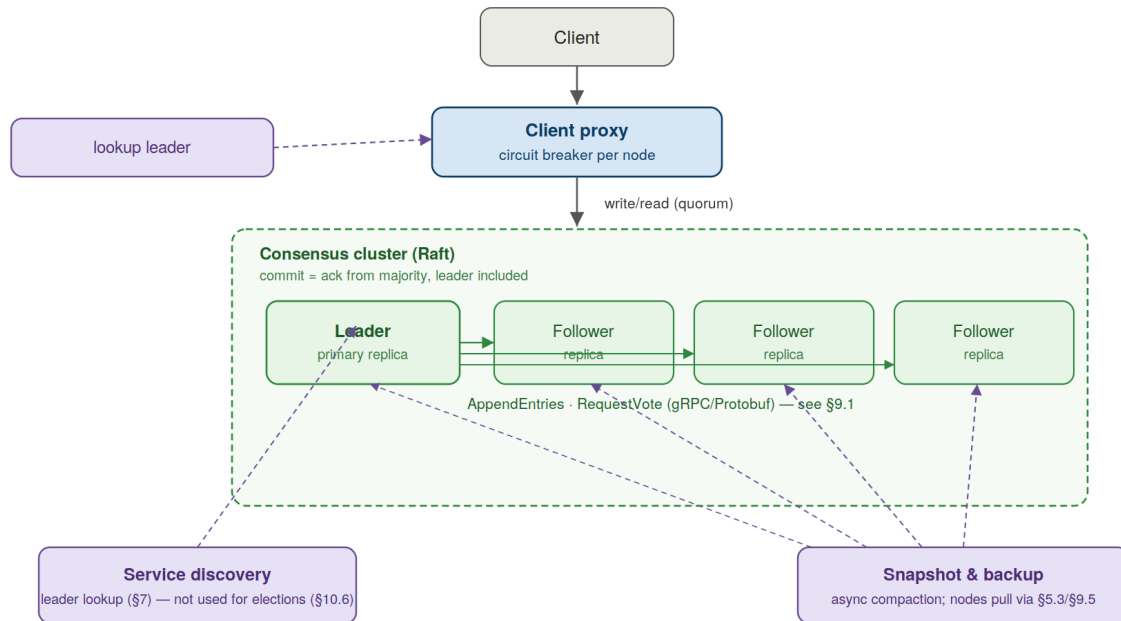
- Exactly three distinct node types must exist as separate deployable services/binaries: Consensus node (stateful), Client proxy service (stateless), Snapshot & backup service (stateless).
- All inter-service and consensus-coordination communication (AppendEntries, RequestVote, GetStatus, and any proxy→consensus-node call) must use gRPC + Protocol Buffers. No REST/HTTP between internal services.
- The external Client → Client Proxy boundary uses REST/JSON (this is the one deliberate exception — see §6).
- At least two architectural patterns must be explicitly implemented and identifiable in the code and in the report: Service Discovery and Circuit Breaker (this specification fixes these two as the chosen patterns; do not substitute them without updating the report).
- The Leader is the primary replica. There is no separate "primary replica" role distinct from "leader" — they are the same process, at the same point in time, for the currently elected term. Do not introduce an extra role layer.

MUST NOT

- No node type may depend on a centralized external database (Postgres, MySQL, Redis, etc.) as its source of truth. Local on-disk files (log segments, snapshots, small metadata files) are fine and expected.
- The Client must never contact a consensus node directly; every request goes through the Client Proxy.
- The Service Discovery component must never sit on the data path (must not process Get/Put requests) and must not add latency to every single key-value operation — it is consulted only for leader/member lookup, at low frequency, with client-side caching.

3. Node type 1 — Consensus node (stateful)

A cluster of nodes maintaining replicated state and coordinating data consistency by implementing the core mechanics of a distributed consensus algorithm (a simplified Raft is the recommended baseline over simplified Paxos, because Raft's roles map directly onto the three required node behaviors and its election/log-replication separation is easier to test incrementally). All consensus nodes run identical code; the only difference between "Leader," "Follower," and "Candidate" is a role field in local runtime state — these are not separate binaries.



3.1 Common node state (persisted to local disk, independent volume per node)

Field	Type	Persisted?	Notes
currentTerm	uint64	yes*	Monotonically increasing election term.
votedFor	string (node id) or empty	yes*	Reset to empty at the start of each new term.
log[]	ordered list of {term, index, command}	yes	Write-ahead log; command is the KV operation (Put/Delete/Update) plus its key/value payload.
commitIndex	uint64	in-memory (recomputed on restart from log + snapshot)	Highest log index known to be committed.
lastApplied	uint64	in-memory	Highest log index applied to the local state machine.
role	enum {Leader, Follower, Candidate}	no	Always starts as Follower on boot.
stateMachine	map[string]string (or []byte)	rebuilt from snapshot + log replay	The actual key-value data.

* Persisted best-effort, not synchronously fsync'd before every RPC reply — see the accepted limitation stated in §4.3. The write path and file format are specified in §3.6.

Log indexing convention: log[] is 1-indexed; index 0 is reserved to mean "no entries" and is never an entry itself. For an empty log, lastLogIndex = 0 and lastLogTerm = 0. This convention must be used consistently in the vote-granting rule (§10.3) and in the log-repair logic (§10.5) to avoid off-by-one errors at the boundary.

3.2 Leader responsibilities (= primary replica)

- Receive a write request forwarded by the Client Proxy (via gRPC, not REST).
- Append the operation to its own local log (write-ahead, on disk, before replicating).
- Replicate the new entry to all followers via AppendEntries RPC.
- Wait for acknowledgements. Commit the operation (and reply to the client) as soon as a majority of nodes (leader included) have acknowledged — not all followers. Quorum size for a cluster of N nodes is $\text{floor}(N/2) + 1$; e.g. with 1 leader + 3 followers (4 nodes total), quorum = 3/4; the system must keep accepting writes even if one follower is unreachable.
- Apply the operation to its local state machine only after commit (never apply before commit).
- Periodically send heartbeats (empty AppendEntries) to followers to maintain leadership and suppress spurious elections.
- If at any point the leader observes a term higher than its own current term — in any RPC request it receives or any RPC reply it gets back from a follower — it must immediately update currentTerm to that higher value, reset votedFor, and revert to Follower before taking any other action (including before finishing whatever it was doing). This generalizes to every role, at every moment, the same rule already stated for Candidates in §3.4: a stale leader that keeps acting as leader after a newer election has already happened is exactly what Election Safety (§10.2) is meant to prevent, and this is the concrete mechanism that enforces it.

3.3 Follower responsibilities

- On receiving AppendEntries: check log consistency (index + term of the previous entry) before appending; if inconsistent, reject and let the leader retry with an earlier index (standard Raft log back-off/repair behavior — see §10.5).
- Append the entry to the local log and persist to disk before acknowledging — an in-memory-only append does not count as durable.
- Send the ack to the leader only after durable persistence.
- Apply committed entries to the local state machine using the leaderCommit field carried in the next AppendEntries call (standard Raft semantics — do not apply speculatively).
- If no heartbeat is received within a randomized election timeout, transition to Candidate and start an election via RequestVote RPCs to all peers.
- When granting a vote to a candidate (§10.3), reset the follower's own election timer at that same moment. Without this, a follower that just voted for someone else may still time out and start its own competing election shortly after, needlessly increasing the split-vote rate.

MUST NOT

- A follower must never answer write requests directly for the Client Proxy. If a follower receives a write meant for the leader, it must respond with a redirect (the current leader's address), never execute it locally.

3.4 Candidate / election responsibilities

- Increment currentTerm, vote for self, reset the election timer, send RequestVote to all peers in parallel.
- Become Leader upon receiving votes from a majority (including self).
- Become Follower again if it discovers a higher term in any RPC response, or if it receives a valid AppendEntries from a legitimate leader for the current term.
- If the election times out with no majority (split vote), start a new election with an incremented term after a new randomized timeout.

3.5 GetStatus (used by Service Discovery)

Every consensus node exposes a lightweight GetStatus() gRPC endpoint returning: current role (leader/follower/candidate), current term, and the node's own network address. This call must be cheap and must never block on log/consensus activity.

3.6 Local persistence layout

This subsection specifies where and how the state listed in §3.1 is actually written to disk, so that the implementing agent has a concrete file layout to build against rather than only the abstract "persisted to local disk" requirement.

- **Volume path:** each consensus-node container mounts an independent Docker volume at `/data/<nodeId>/` (e.g. `/data/node-1/`), consistent with §11's "never share a volume between consensus nodes."
- **Log file** — `/data/<nodeId>/log.dat`: an append-only file of serialized LogEntry records (§9.1), one per Raft log entry. encoding/gob is the recommended encoding for speed of implementation; each record is length-prefixed so a partially-written last record (from a crash mid-append) can be detected and truncated on restart. Every append to this file is followed by an explicit flush/sync before the node acknowledges the corresponding AppendEntries call (§3.3) — this file is what "durable" refers to throughout this document, and it is not covered by the accepted limitation in §4.3.
- **Metadata file** — `/data/<nodeId>/state.json`: a small JSON file holding `{"currentTerm": ..., "votedFor": ...}`, rewritten via write-to-temp-file-then-rename on every change, to avoid leaving a half-written file behind after a crash. Per the accepted limitation stated in §4.3, this write is not required to be synchronously fsync'd before replying to a vote/append RPC — the project accepts the resulting small window of double-voting risk on same-term crash-restart as a documented, deliberate trade-off, rather than treating it as a blocking requirement.
- **Snapshot files** — `/data/<nodeId>/snapshot-<lastIncludedIndex>.dat`: written by the catch-up/compaction flow in §5.3, containing the serialized state-machine map plus lastIncludedIndex/lastIncludedTerm. Once a new snapshot is durably written and log.dat has been truncated up to that index, the previous snapshot file (if any) may be deleted.
- **Startup sequence:** on boot, a node reads the latest snapshot-*.dat (if present) to seed lastApplied and the state machine, then reads state.json for currentTerm/votedFor, then replays log.dat from the snapshot's lastIncludedIndex onward to reconstruct commitIndex/lastApplied and the in-memory log tail.

3.7 Election liveness under partial connectivity (accepted limitation)

A node that is disconnected from a majority of its peers — e.g. isolated by a network partition, or the sole survivor after two simultaneous node failures in a 3-node cluster — will time out and start a new election repeatedly, since by definition it can never observe a heartbeat that would reset its timer. Each such election increments currentTerm, and because Election Safety (§10.2) is enforced purely by term comparison, this node's inflated term forces any legitimate leader it later reconnects with to step down and re-run an election, even though the isolated node itself can never win (its log is guaranteed to be behind, so the vote-granting rule in §10.3 always rejects it once the log-comparison condition is checked). No safety property is violated by this behavior — the isolated node never becomes leader with an incomplete log, and once it reconnects and catches up, it competes for leadership on equal footing with every other node, exactly as § 10.3's rule requires, with no memory of its prior disruptive term inflation.

The standard fix in the Raft literature is the Pre-Vote extension: before incrementing its own term, a node first runs an informal poll asking peers "would you vote for me if I called a real election right now?", and only proceeds to a real, term-incrementing election if that poll suggests it could win. This project does not implement Pre-Vote: it is treated as an accepted limitation rather than a blocking requirement, since the resulting behavior costs only extra, harmless elections (a liveness/availability inefficiency) and never threatens correctness. This trade-off must be stated explicitly in the report's "limitations encountered" section, alongside the accepted limitation already documented in §4.3.

4. Node type 2 — Client proxy service (stateless)

Per the assignment: the input client node that receives read/write requests from outside, dynamically identifies the current cluster Leader, and transparently routes the request.

4.1 Behavior

- Receive a REST/JSON request from the external Client (Get/Put/Update/Delete).
- Consult Service Discovery to learn the current leader and the set of active followers. This call is low-frequency: the proxy keeps a local cache of the cluster map, invalidated only when (a) a circuit breaker for a node trips, or (b) the proxy receives a redirect from a node that is no longer leader.
- Route the request: write (Put/Update/Delete) always to the current Leader only; read (Get) to the Leader by default. Reads may be routed to a follower only through the Read-Index protocol (§4.2), which the project targets implementing from an early milestone since it is the only way a follower read stays strongly consistent. The system ships with an explicit, safe fallback: if Read-Index is not yet complete or is disabled, every read is routed to the Leader with no exceptions — this fallback is always linearizable on its own (§4.1 is the default; §4.2 is the add-on), so the strong-consistency requirement is met regardless of how far Read-Index implementation has progressed.
- Translate the internal gRPC response back into a REST/JSON response for the external client.

MUST NOT

- The proxy never participates in the consensus protocol itself (it must never vote or receive AppendEntries as a cluster member). It is stateless and horizontally replicable without any coordination between proxy instances.

4.2 Linearizable follower reads via the Read-Index protocol

Its purpose is to document, precisely, the only form of follower read that real systems (etcd, TiKV/TiDB) treat as safe: a read served by a follower that is still linearizable, because the follower first confirms with the leader that it is entitled to answer as of the current committed state, instead of answering directly from its own possibly-stale local copy.

- The follower receives a Get request forwarded by the Client Proxy (this requires the proxy's routing policy to allow follower reads at all, which stays off until Read-Index is implemented and validated, per §4.1).
- Before answering, the follower asks the current leader for the leader's latest confirmed commit index ('read index'). The leader must confirm, via a fresh round of heartbeats acknowledged by a quorum, that it is still the legitimate leader before returning this index — this is exactly what prevents a stale or partitioned former leader from handing out a stale read index.
- The follower waits until its own lastApplied index reaches at least the returned read index (i.e. until it has locally applied every entry the leader has confirmed committed).
- Only then does the follower read its local state machine and reply. The result is linearizable: it reflects at least everything committed at the moment the read began, with no request ever executed on the leader's CPU/IO path.
- If the entries the follower needs to catch up on are no longer in the leader's active log because they were already compacted, the follower does not wait on the leader for them: it falls back to the node catch-up flow in §5.3 (Snapshot Catalog API, §9.5) to pull the latest snapshot on its own, then resumes.

- Real systems adopt Read-Index (or a leader lease) for two distinct reasons, which must not be conflated: (a) cutting cross-region network latency, and (b) shedding read load away from the leader onto idle follower capacity. Only reason (a) is a property of large, geographically distributed deployments.
- Reason (a), proximity, does not apply here: this project's consensus nodes all run as Docker Compose services on a single EC2 instance (§11), every node at effectively the same network distance from the Client Proxy. There is no cross-region latency to save by reading from a follower instead of the leader.
- Reason (b), load-shedding, does apply here even on a single host: the leader is still a single OS process handling both the write path (log append + AppendEntries replication) and, by default, every read. Under the scalability scenario in §12.1, concurrent Get traffic still competes with write replication for that one process's CPU/IO — spreading reads across followers via Read-Index genuinely reduces that contention,

independent of geography. This is the motivation for treating Read-Index as a target from early on rather than a deferred nice-to-have.

- Because a safe fallback exists (§4.1: leader-only reads are always linearizable on their own), an incomplete Read-Index implementation is never a correctness risk for the submitted system — at worst it is a missed performance optimization, which should be stated plainly as a limitation in the report if it doesn't get finished.

4.3 Persistence of consensus state (accepted limitation)

Raft requires every node to persist to disk, before responding to any RPC, at least two fields: `currentTerm` and `votedFor` (in addition to the log itself, which is synchronously persisted — see §3.6). In the version implemented for this project, synchronous persistence of `currentTerm/votedFor` specifically is treated as an accepted limitation rather than a blocking requirement for delivery: in case of a crash and restart of a node within the same term, there is a theoretical risk of double voting that the project does not fully eliminate. This choice is justified by the academic scope and delivery timeline of the project, and must be stated explicitly in the report, in the "limitations encountered" section, as a conscious trade-off between formal Raft correctness and a sustainable implementation effort for a single student.

5. Node type 3 — Snapshot & backup service (stateless)

Per the assignment: an asynchronous service responsible for querying consensus nodes to download and compact historical logs into stable checkpoint files.

5.1 Behavior

- Query Service Discovery to learn the current leader (never hard-code a leader address — the leader changes over time after elections).
- Periodically poll the leader for log occupancy/size.
- When occupancy reaches a configurable threshold (default recommendation: 30%), start compaction: download the oldest entries (via `StreamLogRange`, §9.4 — this download may target any node, leader or follower, since the entries are already committed), transfer them to compact backup storage, and only remove them from the leader's active log after the backup write is confirmed.
- Concurrency guard: if, while backing up, the service detects that occupancy has reached a second, higher threshold (default recommendation: 90%) — meaning another compaction cycle appears to already be in flight — it must wait for that operation to finish and for the corresponding entries to be removed from the leader before proceeding, to avoid double-compaction corruption.
- Before calling `ConfirmCompaction`, re-verify via Service Discovery that the target node is still the current leader. If leadership changed during this cycle, abort without truncating anything and restart the cycle against the new leader — no data is at risk, since Leader Completeness (§10.2) guarantees the new leader already holds every entry that was safe to compact.
- Entries removed from a node's active log remain available to any node that later needs them — a lagging follower or a brand-new node alike — exclusively through the node-initiated catch-up flow in §5.3 (Snapshot Catalog API, §9.5). The Leader never pushes snapshots to followers directly; §5.3 is the single, sole mechanism for this in the system.

5.2 Snapshot contents

A snapshot must include not just the truncated log, but also the applied state of the state machine (the resulting key-value map) at that point. This lets a node that rejoins the cluster after an absence reconstruct state by loading the snapshot and replaying only the subsequent log entries, instead of replaying the entire log from scratch.

MUST NOT

- The snapshot service never writes directly to the leader's log, never participates in voting/elections, and is not on the critical write path: its own unavailability must never block key-value operations.

5.3 Node catch-up and per-node log compaction (periodic snapshot polling)

This subsection specifies how every consensus node — not only a newly-created follower — stays synchronized with the compaction performed above, reusing the same periodic-polling pattern already used by Service Discovery (§7) for architectural consistency. This is the system's only catch-up mechanism — there is no separate leader-initiated push RPC for this purpose (see §5.1).

- Every consensus node, on a periodic timer (configurable interval, default recommendation: 5s — see §10.1), calls `GetLatestSnapshotInfo` on the Snapshot & Backup service's Snapshot Catalog API (§9.5) to learn the `lastIncludedIndex` of the most recent stable snapshot.
- Catch-up rule: if the returned `lastIncludedIndex` is greater than the node's own current `lastApplied`, the node calls `FetchSnapshot`, applies the received state as its new state-machine baseline, and anchors its local log by creating a single placeholder entry at `{lastIncludedIndex, lastIncludedTerm}`. This anchor exists only to satisfy the `prevLogIndex/prevLogTerm` consistency check on subsequent `AppendEntries` calls — it is never used to answer a client read directly. After anchoring, the node resumes normal `AppendEntries`-based replication with its leader for any entries beyond the snapshot.
- This same rule covers both cases uniformly: a brand-new node with no local state at all, and an already-attached follower that has simply fallen behind a compaction point while connected. No separate code path is needed for the two situations.
- Failure handling: if the `FetchSnapshot` call itself fails (transient network or service unavailability), the node must not discard its existing local state or restart from zero. It keeps whatever log/state it already has and simply retries at the next periodic interval.

- Local compaction rule, applying to every node including the leader: if a node's own lastApplied is already greater than or equal to the latest lastIncludedIndex reported by the Snapshot & Backup service, that node may safely truncate its own local log up to that index, since a durable copy already exists in the backup store and can be recovered later if needed. This is the mechanism that guarantees log compaction actually happens on every node over time — not only on whichever single node the Snapshot & Backup service happened to download from during its own compaction cycle (§5.1).

- Because this check runs on a periodic interval rather than being triggered synchronously by the leader, there is a bounded window — at most one polling interval plus snapshot transfer time — during which a lagging node cannot serve as a useful AppendEntries target for the newest entries. This does not threaten correctness: a node that has not caught up also cannot win a leader election, by the vote-granting rule in §10.3.

6. Client (external actor)

Not one of the three graded node types, but part of the system's external contract.

- Sends REST requests to the Client Proxy with parameters as JSON (push/get/update/delete — map these to Put/Get/Update/Delete internally).
- Has no knowledge of cluster topology: does not know who the leader is, how many consensus nodes exist, and does not implement low-level retries (that responsibility belongs to the Client Proxy).
- Must never contact a consensus node directly.

REST	Metodo HTTP	Endpoint	RPC gRPC
push	POST	/v1/kv/{key}	Put
update	PUT	/v1/kv/{key}	Update
get	GET	/v1/kv/{key}	Get
delete	DELETE	/v1/kv/{key}	Delete

GET /healthz → 200 OK liveness only for the proxy not the status of all the cluster

Note — Put vs. Update semantics. §9.2 defines **Put** and **Update** as two distinct RPCs with "the same shape... semantically an overwrite," but does not specify whether **Update** on a nonexistent key should fail or behave as an upsert. As implemented and verified against a running cluster, both **Put** (push, **POST**) and **Update** (**PUT**) perform an unconditional upsert against the state machine — repeating either on the same key simply overwrites the value, and calling **Update** on a key that has never been written creates it rather than returning an error. The two RPCs are therefore functionally interchangeable in this implementation; the distinction exists at the REST/RPC contract level (§6, §9.2) for semantic clarity to the client, not as a behavioral difference enforced by the state machine. This should be stated explicitly in the report as a documented design clarification, since the specification alone left it ambiguous.

7. Service Discovery (architectural pattern 1)

Maintains a dynamic, up-to-date view of consensus cluster membership and of the current leader. This satisfies the assignment's explicit Service Discovery pattern requirement.

- Every consensus node exposes GetStatus() via gRPC (role, term, address).
- The Service Discovery component performs periodic polling of all known nodes. Node bootstrap is via static initial configuration (e.g., a list of addresses in a config file or environment variables) — an external system

such as Consul or etcd is not required and its omission should be explicitly justified in the report as a scope decision appropriate to this project's size.

- It exposes its own API, queried by the Client Proxy and by the Snapshot & Backup service, returning: current leader address, list of active followers.

MUST NOT

- Service Discovery is never on the data path: it never receives application read/write requests, and must not add latency to key-value traffic. It is queried only for lookup, sparingly and cacheably, never once per KV request.

8. Circuit Breaker (architectural pattern 2)

Implemented in the Client Proxy: one breaker instance per outbound connection to a known cluster node (leader and each known follower).

- If calls to a given node fail repeatedly beyond a configurable threshold, that node's breaker opens; the proxy stops routing to it and immediately requests a refreshed cluster map from Service Discovery.
- This prevents cascading retries during an ongoing election or a network fault.
- Recommended library: a mature open-source Go circuit breaker (e.g. sony/gobreaker) rather than a hand-rolled implementation, to save implementation time — this is a supporting library, not the originality component of the project.

Parametro gobreaker	Valore consigliato	Perché questo valore
ReadyToTrip (consecutive failures before opening)	3 consecutive failures	At RPC timeout = 100ms (§10.1), 3 failures $\approx$ 300ms — roughly the same window as the election-timeout range (150–300ms, §10.1), so the breaker opens on about the same timescale the cluster would already be re-electing on. Neither trigger-happy on a single blip nor slow to stop hammering a dead node.
Timeout (open-state duration before probing again)	2s	Covers a full election + heartbeat-convergence cycle with margin, and sits close to the Service Discovery polling interval, so by the time the breaker probes again the cluster view has likely already been refreshed.
MaxRequests (probe requests allowed in half-open state)	1	A single probe request; if the node is still down, no reason to risk further traffic before re-opening.
Interval (failure-count reset while closed)	30s	Forgets isolated old failures after half a minute, so a node with a transient issue long ago isn't flagged as suspect indefinitely. Not safety-critical — 0 (never reset except on success) is an equally defensible, simpler alternative to state in the report.

9. RPC / Protobuf contract (authoritative)

This is the canonical interface surface. Any implementation must expose (at minimum) these RPCs with these exact semantics; field names may be adapted to Go naming conventions but the shape and meaning must be preserved.

9.1 Consensus-internal service (consensus.proto)

```
service Consensus {
  rpc RequestVote (RequestVoteRequest) returns (RequestVoteReply);
  rpc AppendEntries (AppendEntriesRequest) returns (AppendEntriesReply);
  rpc GetStatus (GetStatusRequest) returns (GetStatusReply);
}

message RequestVoteRequest {
  uint64 term = 1;
  string candidateId = 2;
  uint64 lastLogIndex = 3;
  uint64 lastLogTerm = 4;
}

message RequestVoteReply {
  uint64 term = 1;
  bool voteGranted = 2;
}

message LogEntry {
  uint64 term = 1;
  uint64 index = 2;
  bytes command = 3; // serialized KVCommand (see below)
}

message AppendEntriesRequest {
  uint64 term = 1;
  string leaderId = 2;
  uint64 prevLogIndex = 3;
  uint64 prevLogTerm = 4;
  repeated LogEntry entries = 5; // empty => heartbeat
  uint64 leaderCommit = 6;
}

message AppendEntriesReply {
  uint64 term = 1;
  bool success = 2;
  // Optional but recommended for fast log back-off:
  uint64 conflictIndex = 3;
  uint64 conflictTerm = 4;
}

enum Role { FOLLOWER = 0; CANDIDATE = 1; LEADER = 2; }

message GetStatusRequest {}
message GetStatusReply {
  Role role = 1;
  uint64 term = 2;
  string address = 3;
  string nodeId = 4;
}
```

9.2 Client-facing KV service (proxy → leader, gRPC internally)

```
service KVService {
  rpc Get (GetRequest) returns (GetReply);
  rpc Put (PutRequest) returns (WriteReply);
  rpc Update (PutRequest) returns (WriteReply); // same shape as Put, semantically an overwrite
  rpc Delete (DeleteRequest) returns (WriteReply);
}

message GetRequest { string key = 1; }
message GetReply { bool found = 1; string value = 2; string redirectLeader = 3; }
```

```

message PutRequest    { string key = 1; string value = 2; }
message DeleteRequest { string key = 1; }

message WriteReply {
  bool success = 1;
  string redirectLeader = 2; // set when this node is not the leader
  uint64 commitIndex = 3;    // for testing/observability
}

// Internal representation appended to the Raft log:
message KVCommand {
  enum Op { PUT = 0; DELETE = 1; UPDATE = 2; }
  Op op = 1;
  string key = 2;
  string value = 3; // empty for DELETE
}

```

9.3 Service Discovery API (queried by Client Proxy & Snapshot service)

```

service Discovery {
  rpc GetClusterView (Empty) returns (ClusterView);
}
message NodeInfo { string nodeId = 1; string address = 2; Role role = 3; uint64 term = 4; }
message ClusterView {
  string leaderAddress = 1;
  repeated NodeInfo followers = 2;
  repeated NodeInfo allNodes = 3;
}

```

9.4 Snapshot & Backup internal RPC (backup service ↔ leader or a follower — see §5.1)

```

service SnapshotTransfer {
  rpc GetLogStatus (Empty) returns (LogStatusReply);
  rpc StreamLogRange (LogRangeRequest) returns (stream LogEntry);
  rpc ConfirmCompaction (CompactionAck) returns (Empty);
}
message LogStatusReply { uint64 totalEntries = 1; uint64 occupancyPercent = 2; uint64 firstIndex = 3; uint64 lastIndex = 4; }
message LogRangeRequest { uint64 fromIndex = 1; uint64 toIndex = 2; }
message CompactionAck { uint64 truncatedUpToIndex = 1; bytes stateMachineSnapshot = 2; }

```

- StreamLogRange (the bulk historical download) may target any node, leader or follower: the entries it reads are by definition already committed, so any node's durable copy is equally valid — no linearizability concern applies, unlike client reads (§4.1/§4.2). GetLogStatus and ConfirmCompaction, by contrast, must stay bound to the current leader, since only the leader can authoritatively decide what is safe to truncate right now. Before calling ConfirmCompaction, the service re-verifies via Service Discovery that its target is still the current leader; if leadership changed mid-cycle, it aborts this cycle without truncating anything and restarts against the new leader — no data is at risk, since Leader Completeness (§10.2) guarantees the new leader already holds every entry that was safe to compact.

9.5 Snapshot Catalog API (queried by any consensus node — see §5.3)

Exposed by the Snapshot & Backup service so that any consensus node — a brand-new node, or an existing follower or leader that has fallen behind — can independently check for and retrieve the latest stable snapshot, without needing the leader to drive the process. This is the system's sole snapshot-delivery path (see §5.1, §5.3).

```

service SnapshotCatalog {
  rpc GetLatestSnapshotInfo (Empty) returns (SnapshotInfo);
  rpc FetchSnapshot (Empty) returns (stream SnapshotChunk);
}
message SnapshotInfo {
  uint64 lastIncludedIndex = 1;
  uint64 lastIncludedTerm = 2;
}

```

```
    bool    available          = 3;
}
message SnapshotChunk {
    bytes data    = 1; // chunked state-machine + log-anchor payload
    bool  isLast = 2;
}
```

- Note for the implementing agent: the exact wire format above is a specification baseline, not a rigid mandate on message numbering — field numbers may be renumbered as needed for actual .proto compilation, but every listed RPC and field's semantic role must be present somewhere in the implementation.

10. Consensus algorithm specification (Raft-lite)

Recommended scope: implement a minimal-but-correct subset of Raft — leader election, log replication, commit rule, and basic crash recovery via persisted state and snapshots. Do not implement dynamic cluster membership changes (joint consensus) — the cluster's consensus-node set is fixed at deploy time via Docker Compose. Do not implement multi-Raft/sharding.

10.1 Timing parameters (must be configurable, e.g. via env vars)

Parameter	Suggested default	Notes
Election timeout	random in [150ms, 300ms]	Randomization is required to avoid repeated split votes.
Heartbeat interval	50ms	Must be well below the minimum election timeout.
RPC timeout	100ms (tunable per experiment)	Used in the scalability/fault-tolerance measurements — see §12.
Snapshot poll interval	5s (configurable)	Used by every node to check for new snapshots and to safely compact its own local log — see §5.3.
Service discovery periodic polling	2s(configurable)	Used by the service discovery for discover where the consensus nodes are and pool the request.

10.2 Safety properties the implementation must uphold

- Election Safety: at most one leader per term.
- Leader Append-Only: the leader never overwrites or deletes entries in its own log, only appends.
- Log Matching: if two logs contain an entry with the same index and term, all preceding entries are identical.
- Leader Completeness: a committed entry will be present in the logs of leaders of all higher terms.
- State Machine Safety: if a node has applied a log entry at a given index, no other node will ever apply a different entry for that same index.

10.3 Vote-granting rule (RequestVote)

A node grants its vote to a candidate only if both conditions hold: (1) it has not already voted for a different candidate in the candidate's term (votedFor is empty, or already equals this candidate, for that term); and (2) the candidate's log is at least as up-to-date as the voter's own log — compared first by lastLogTerm (higher wins), and only if equal, by lastLogIndex (higher or equal wins), using the indexing convention fixed in §3.1. If either condition fails, the node rejects the vote (voteGranted = false) and returns its own current term, which lets a stale candidate step down. Granting a vote also resets the voter's own election timer (§3.3), to avoid it starting a redundant competing election immediately afterward.

10.4 Commit rule: entries from the leader's current term only

A leader may advance commitIndex only when the entry at that index was appended during its own current term. It must never commit an entry directly on the basis of quorum replication alone if that entry belongs to an earlier term — such older entries are only committed indirectly, once a later entry from the leader's own term has itself been replicated to a majority and becomes committed (which implicitly commits everything before it in the log). Omitting this rule is the most common correctness bug in hand-written Raft implementations — it corresponds to the "Figure 8" scenario in the original Raft paper — and can cause a committed entry to be silently overwritten after a cascading leader crash.

10.5 Log repair after a rejected AppendEntries (nextIndex / matchIndex)

Each leader maintains, in memory, nextIndex[] and matchIndex[] for every follower. On a successful AppendEntries, the leader sets matchIndex[follower] to the index of the last entry sent and nextIndex[follower] to the following index. On a rejection, the leader must not simply decrement nextIndex by one and retry, since this converges too slowly when many entries conflict; instead it uses the conflictIndex/conflictTerm fields returned by the follower (§9.1) to jump nextIndex directly to the first entry of the conflicting term, skipping the rest of that term's entries in a single round-trip.

10.6 Peer address configuration for the consensus layer

Consensus nodes must not depend on the Service Discovery service, or on any other runtime call, to learn the addresses of their peers for RequestVote/AppendEntries — doing so would make leader election depend on the availability of a component that is explicitly kept out of the critical path (§7). Instead, every consensus node receives the same static peer list used to bootstrap Service Discovery (§7), via the same configuration mechanism (e.g. one shared environment variable or config file mounted identically into both the Service Discovery container and every consensus-node container). Since cluster membership is fixed at deploy time (§17), this list never needs to change at runtime.

10.7 Client Retry Semantics.

The Raft consensus layer guarantees that all nodes agree on the same total order of committed log entries, but it does not by itself guarantee exactly-once execution from the client's perspective: if a client receives a timeout or an ErrLeadershipLost response after proposing a command, it cannot distinguish between "the entry was never appended" and "the entry was appended and committed, but the leader crashed before replying." The standard solution described in the original Raft paper (§8, "Client interaction") is to have the client retry the same command against the (possibly new) leader, and to push deduplication down to the application layer.

A deduplication mechanism, in this context, means tagging each client command with a unique identifier — typically a pair (client ID, sequence number) — and having the state machine keep a small table of the last identifier it has applied for each client. When an entry is applied, the state machine first checks this table: if the identifier has already been seen, the entry is skipped and the previously computed result is returned as if it had just been applied; otherwise, it is applied normally and the table is updated. This turns an at-least-once delivery guarantee (from client retries) into an exactly-once execution guarantee (from the state machine's point of view), independently of how many times the underlying Raft log actually contains the same command.

In this project, we deliberately avoid implementing this deduplication mechanism, since it would add complexity without providing any additional correctness guarantee for our workload. The system exposes a key-value store whose fundamental write operations are SET key=value and DELETE key: both are naturally idempotent, as applying either one once or multiple times in sequence yields the same final state (a repeated SET leaves the key at the same value; a repeated DELETE leaves the key absent). A client retry that causes the same entry to be appended and applied twice (or more) therefore does not compromise the consistency of the replicated state; the log may transiently contain duplicate entries, but the resulting key-value mapping converges to the same value regardless. We consider this an intentional simplification enabled by the semantics of the workload rather than a limitation of the consensus layer itself, and we distinguish it explicitly from operations that are not naturally idempotent (e.g., increment/decrement commands), which — although out of scope for this project — would require the client-ID + sequence-number deduplication scheme described above.

10.8 Recommended build/test order (for the implementing agent)

- Implement the state machine and log data structures with unit tests, no networking at all.
- Implement leader election and log replication logic against an in-process fake transport (Go channels simulating RPC, deterministic scheduling) to validate protocol correctness cheaply before touching gRPC.
- Only after in-process tests pass, wire the same logic to real gRPC calls between separate OS processes / containers.
- Add persistence (write-ahead log to disk, term/votedFor to disk per §3.6) and crash-recovery tests (kill -9 a process, restart it, verify it rejoins correctly).
- Add snapshotting last, once basic replication and elections are verified stable.

10.9 gRPC transport configuration (passthrough resolver, waitForReady)

During implementation, a real correctness gap was found in the gRPC client used for peer-to-peer calls (RequestVote, AppendEntries, RequestReadIndex, and the Snapshot & Backup service's own client toward consensus nodes): gRPC-Go's **default resolver scheme is dns**, not **passthrough**. With a bare **host:port** target, the client caches DNS resolution and does not reliably re-resolve a peer's hostname after that peer's container is killed and restarted, since Docker may assign it a new IP address on the bridge network. In practice this surfaced as a leader that kept failing to reach a just-restarted peer with "**name resolver error: produced zero addresses**", sometimes for minutes, well after the peer's container was fully up and reachable.

The fix, which every gRPC client created for inter-service communication in this project must apply consistently:

- Dial every peer as **passthrough:///<host>:<port>** instead of a bare target, so every reconnection attempt re-resolves via a fresh **net.Dial** against Docker's embedded DNS, instead of going through gRPC-Go's own cached resolver.
- Pass **grpc.WaitForReady(true)** as a call option on RPCs that should tolerate a brief post-restart connect gap (**AppendEntries**, **ConfirmCompaction**, **StreamLogRange**, **FetchSnapshot**) — this lets the call wait for the automatic backoff/reconnect (already provided by gRPC-Go by default) to complete, instead of failing immediately with **Unavailable**.
- Deliberately **omit WaitForReady** on **RequestVote** and on cheap, frequent polling calls (**GetLogStatus**, **GetStatus**, **GetLatestSnapshotInfo**): fail-fast is correct there, since a candidate collecting votes must not block on a single unreachable peer, and a failed poll simply retries at the next interval.

This is not a workaround specific to one code path — every new gRPC client added later in the project (e.g. for the Snapshot Catalog API, §9.5) must reuse the same dialing pattern rather than reintroducing the bug independently.

11. Deployment: Docker Compose & EC2

- Orchestration via Docker Compose; each node type is a separate image/service definition. All services run on a single Amazon EC2 instance (a small instance, e.g. t3.small/t3.medium, is sufficient for the required scenarios), consistent with the assignment text and with the Read-Index proximity argument in §4.2.
- Recommended compose topology: 1 Service Discovery instance, N consensus nodes (N configurable, e.g. 3/5/7 for the scalability scenario), 1+ client proxy instances, 1 snapshot & backup instance.
- Each consensus node runs in its own container (never more than one node per container), with its own independent persistent volume for its on-disk log/state (§3.6) — never share a volume between consensus nodes.
- All containers are connected through a dedicated local Docker network (bridge network), not exposed externally. Within this network, only the Client Proxy is exposed externally: no consensus node and no Service Discovery instance is directly reachable from the Client.
- Circuit Breaker requires no separate container: it is a software component integrated inside the Client Proxy (one breaker instance per connection to each known cluster node), not a standalone service.
- Storage: local on-disk files per node as specified in §3.6 — there is no separate storage/database service or container (see §2 MUST NOT).
- The static peer-address list used for consensus-node elections (§10.6) and for Service Discovery bootstrap (§7) should be defined once (e.g. one shared .env file or Compose extension field) and mounted identically into every relevant container, to avoid configuration drift between the two.

12. Evaluation scenarios to implement

12.1 Scalability scenario

- Vary the number of consensus nodes (e.g. 3, 5, 7).
- Measure the impact on RPC response times — specifically end-to-end latency of writes (proxy → leader → AppendEntries → quorum ack → reply) as the quorum size grows.
- Use a small Go load generator hitting the Client Proxy, recording latency percentiles (p50/p95/p99) per cluster size.

12.2 Fault-tolerance scenario

- Induce a sudden crash of the current Leader (e.g. docker stop or kill -9 on that container).
- Measure: (a) time to detect the failure (election timeout elapsed), (b) time to elect a new leader, (c) time until the state has converged and the system is serving client requests correctly again.
- This scenario is well-defined precisely because Leader and primary replica are a single node — there is no ambiguity about which process is actually being killed.

- Both scenarios' raw results (latency numbers, timing traces, logs with timestamps) should be captured into plots/tables and included in the report (§15), plus the underlying scripts kept in the repository under experiments/ or similar.

13. Suggested repository layout

```
b5-kvstore/
├── proto/                # .proto source files (consensus, kv, discovery, snapshot)
├── cmd/
│   ├── consensus-node/  # main.go for the consensus node binary
│   ├── client-proxy/    # main.go for the proxy binary
│   ├── snapshot-backup/ # main.go for the snapshot/backup binary
│   └── discovery/       # main.go for the service discovery binary
├── internal/
│   ├── raft/            # election, log replication, persistence, safety logic
│   ├── statemachine/    # KV map + apply logic
│   ├── proxy/           # routing, redirect handling, REST<->gRPC translation
│   ├── discovery/       # polling, cluster view aggregation
│   ├── circuitbreaker/  # breaker wiring around proxy->node calls
│   └── snapshot/        # compaction logic, thresholds, transfer
├── deployments/
│   ├── docker-compose.yml
│   └── ec2/             # provisioning notes/scripts
├── experiments/
│   ├── scalability/     # load generator + result scripts/plots
│   └── fault-tolerance/ # crash-injection scripts + timing analysis
├── report/              # LaTeX (ACM/IEEE template), max 5 pages
├── README.md
└── go.mod
```

14. Implementation checklist / acceptance criteria

#	Criterion	Definition of done
1	Three node types deployable independently	Three separate Docker images/services start and run via docker-compose up.
2	gRPC/Protobuf internal comms	No REST between internal services; only Client↔Proxy is REST/JSON.
3	Leader election	Killing the leader results in a new leader within a bounded, measured time; only one leader per term at any time.
4	Log replication & quorum commit	Writes commit and are visible after ack from majority, even with one follower down.
5	Durable log persistence	Restarting a killed node reloads log/state from disk per §3.6 and rejoins correctly (subject to the accepted limitation in §4.3 for currentTerm/votedFor).
6	Client Proxy routing + redirect	Writes sent to a non-leader are redirected transparently; proxy never exposes this to the external client.
7	Service Discovery	Proxy and snapshot service always resolve the current leader correctly after an election, without hard-coded addresses.
8	Circuit Breaker	Repeated failures to a node open its breaker and trigger a forced discovery refresh; verified via an integration test that kills a follower.
9	Snapshot & backup	Compaction triggers at the configured threshold, truncates the log only after confirmed backup write, and a rejoining node can rebuild state from snapshot + replay.
10	Scalability experiment	Latency data collected for at least 3 different cluster sizes, with plots/tables in the report.
11	Fault-tolerance experiment	Leader-crash timing data (detection + election + convergence) collected and reported.
12	EC2 deployment	The full Compose stack has been run on an actual EC2 instance at least once, with evidence (screenshots/logs) kept for the report/demo.
13	Vote-granting safety	A test where a candidate has a stale or shorter log never wins an election over a node with a more up-to-date log, per the rule in §10.3.
14	Commit-safety across leader changes	Repeated forced leader crashes during in-flight writes never lose or overwrite a committed entry, per the commit rule in §10.4.
15	Node catch-up & per-node compaction	A node started from empty state (or rolled back) bootstraps correctly via the Snapshot Catalog API (§5.3) and resumes replication; every node's on-disk log is confirmed to shrink over time under sustained write load.

15. Deliverables & submission

- **Deadline:** September 18, 2026, via email to the instructor.

Contents of the submission:

- Commented source code (link to a cloud storage space or repository, e.g. Google Drive or GitHub).
- A report in PDF format, max 5 pages, recommended in scientific-article style using the ACM or IEEE double-column proceedings template ([ACM template](#) / [IEEE template](#)), covering: system architecture and design choices with justification; the actual implementation; software platform used for development (incl. supporting libraries) and application testing; test results; any limitations encountered (including the accepted limitation in §4.3, and the Read-Index completion status per §4.1/§4.2).
- A short README file for installation, configuration, and execution.

16. Grading criteria

- Compliance with requirements.
- Originality.
- Test results.
- Code and report organization and readability.
- Quality and clarity of the presentation, including answers to the instructor's questions during discussion.

For further general information, refer to the project-presentation slides available on the course's Microsoft Teams channel for "Sistemi Distribuiti e Cloud Computing."

17. Explicit non-goals / scope cuts (deliberate, to keep the project deliverable within available time)

- No dynamic cluster membership changes (no joint consensus, no add/remove-node RPCs) — cluster size is fixed at deploy time.
- No external service-discovery system (Consul/etcd) — a small home-grown polling registry is sufficient and must simply be justified in the report.
- Follower reads are not the default policy (see §4.1): reads are served by the Leader unless the Read-Index protocol in §4.2 is complete and enabled. A bare "read from the nearest follower" without Read-Index must never be presented as consistency-preserving, since with all nodes co-located on one EC2 host (§11) there is no real proximity advantage to justify it.
- No multi-Raft / sharding / partitioning of the keyspace across multiple independent consensus groups.
- No per-node EC2 instances: all services run on a single EC2 instance via Docker Compose (§11), not one instance per consensus node — the latter would also invalidate the Read-Index proximity argument in §4.2.
- A mature third-party consensus library (e.g. hashicorp/raft) should be treated as a last-resort fallback only, since the assignment expects the core consensus mechanics to be implemented by the student; using one may weigh negatively on the "originality" and "requirement compliance" criteria.

18. References

- L. Lamport. "Paxos Made Simple." ACM SIGACT News, 32(4):18–25, 2001.
<https://lamport.azurewebsites.net/pubs/paxos-simple.pdf>
- D. Ongaro and J. Ousterhout. "In Search of an Understandable Consensus Algorithm." Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC'14), pages 305–320, 2014.
<https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf>